Designing Lambda applications

A well-architected event-driven application uses a combination of AWS services and custom code to process and manage requests and data. This chapter focuses on Lambda-specific topics in application design. There are many important considerations for serverless architects when designing applications for busy production systems.

Many of the best practices that apply to software development and distributed systems also apply to serverless application development. The overall goal is to develop workloads that are:

- **Reliable** – offering your end users a high level of availability. AWS serverless services are reliable because they are also designed for failure.
- **Durable** – providing storage options that meet the durability needs of your workload.
- **Secure** – following best practices and using the tools provided to secure access to workloads and limit the blast radius.
- **Performant** – using computing resources efficiently and meeting the performance needs of your end users.
- **Cost-efficient**– designing architectures that avoid unnecessary cost that can scale without overspending, and also be decommissioned without significant overhead.

The following design principles can help you build workloads that meet these goals. Not every principle may apply to every architecture, but they should guide you in general architecture decisions.

Topics

Use services instead of custom code

Serverless applications usually comprise several AWS services, integrated with custom code run in Lambda functions. While Lambda can be integrated with most AWS services, the services most commonly used in serverless applications are:

Category	AWS service
Compute	AWS Lambda
Data storage	Amazon S3 Amazon DynamoDB Amazon RDS
API	Amazon API Gateway
Application integration	Amazon EventBridge Amazon SNS Amazon SQS
Orchestration	Lambda durable functions AWS Step Functions
Streaming data and analytics	Amazon Data Firehose

> **Note**
>
> Many serverless services provide replication and support for multiple Regions, including DynamoDB and Amazon S3. Lambda functions can be deployed in multiple Regions as part of a deployment pipeline, and API Gateway can be configured to support this configuration. See this example architecture that shows how this can be achieved.

There are many well-established, common patterns in distributed architectures that you can build yourself or implement using AWS services. For most customers, there is little commercial value in

investing time to develop these patterns from scratch. When your application needs one of these patterns, use the corresponding AWS service:

Pattern	AWS service
Queue	Amazon SQS
Event bus	Amazon EventBridge
Publish/subscribe (fan-out)	Amazon SNS
Orchestration	Lambda durable functions AWS Step Functions
API	Amazon API Gateway
Event streams	Amazon Kinesis

These services are designed to integrate with Lambda and you can use infrastructure as code (IaC) to create and discard resources in the services. You can use any of these services via the AWS SDK without needing to install applications or configure servers. Becoming proficient with using these services via code in your Lambda functions is an important step to producing well-designed serverless applications.

Understand Lambda abstraction levels

The Lambda service limits your access to the underlying operating systems, hypervisors, and hardware running your Lambda functions. The service continuously improves and changes infrastructure to add features, reduce cost and make the service more performant. Your code should assume no knowledge of how Lambda is architected and assume no hardware affinity.

Similarly, Lambda's integrations with other services are managed by AWS, with only a small number of configuration options exposed to you. For example, when API Gateway and Lambda interact, there is no concept of load balancing since it is entirely managed by the services. You also have no direct control over which Availability Zones the services use when invoking functions at any point in time, or how Lambda determines when to scale up or down the number of execution environments.

This abstraction helps you focus on the integration aspects of your application, the flow of data, and the business logic where your workload provides value to your end users. Allowing the services to manage the underlying mechanics helps you develop applications more quickly with less custom code to maintain.

Implement statelessness in functions

For standard Lambda functions, you should assume that the environment exists only for a single invocation. The function should initialize any required state when it is first started. For example, your function may require fetching data from a DynamoDB table. It should commit any permanent data changes to a durable store such as Amazon S3, DynamoDB, or Amazon SQS before exiting. It should not rely on any existing data structures or temporary files, or any internal state that would be managed by multiple invocations.

When using Durable Functions, state is automatically preserved between invocations, eliminating the need to manually persist state to external storage. However, you should still follow stateless principles for any data not explicitly managed through the DurableContext.

To initialize database connections and libraries, or load state, you can take advantage of static initialization. Since execution environments are reused where possible to improve performance, you can amortize the time taken to initialize these resources over multiple invocations. However, you should not store any variables or data used in the function within this global scope.

Minimize coupling

Most architectures should prefer many, shorter functions over fewer, larger ones. The purpose of each function should be to handle the event passed into the function, with no knowledge or expectations of the overall workflow or volume of transactions. This makes the function agnostic to the source of the event with minimal coupling to other services.

Any global-scope constants that change infrequently should be implemented as environment variables to allow updates without deployments. Any secrets or sensitive information should be stored in AWS Systems Manager Parameter Store or AWS Secrets Manager and loaded by the function. Since these resources are account-specific, you can create build pipelines across multiple accounts. The pipelines load the appropriate secrets per environment, without exposing these to developers or requiring any code changes.

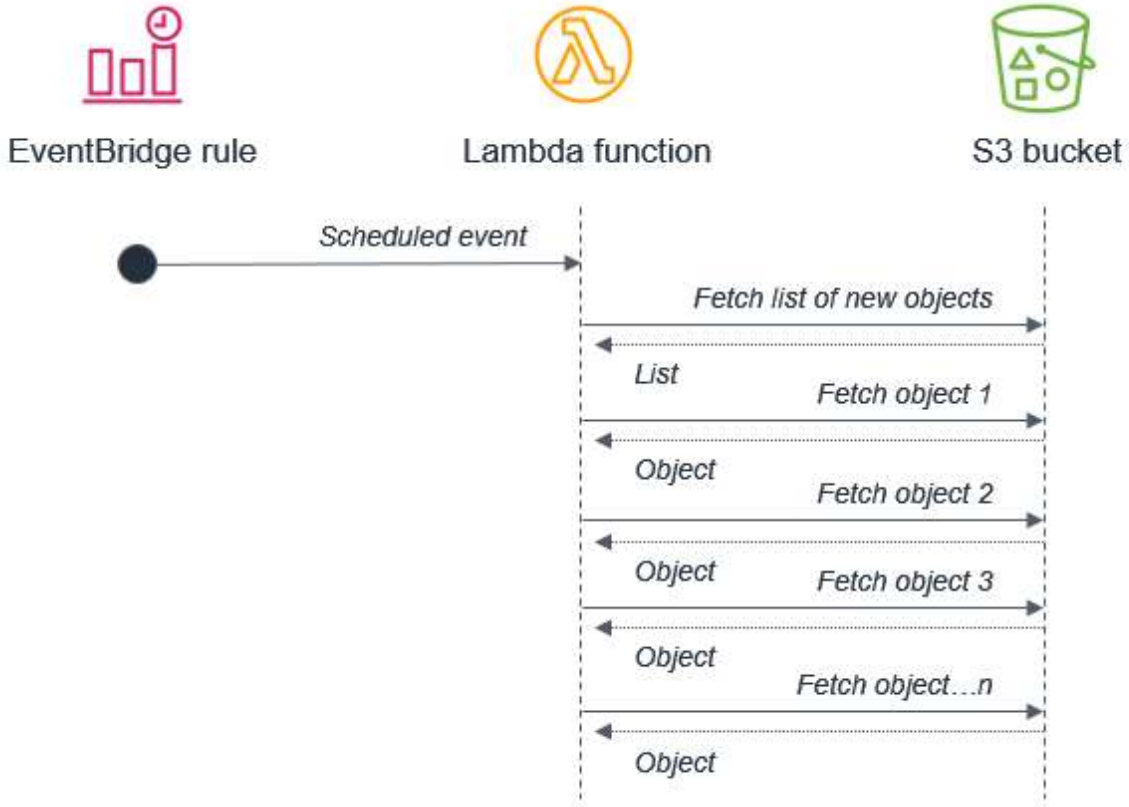
Build for on-demand data instead of batches

Many traditional systems are designed to run periodically and process batches of transactions that have built up over time. For example, a banking application may run every hour to process ATM transactions into central ledgers. In Lambda-based applications, the custom processing should be triggered by every event, allowing the service to scale up concurrency as needed, to provide near-real time processing of transactions.

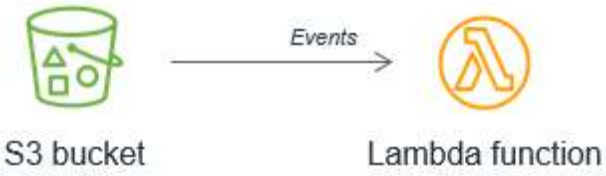
While standard Lambda functions are limited to 15 minutes of execution time, Durable Functions can run for up to one year, making them suitable for longer-running processing needs. However, you should still prefer event-driven processing over batch processing when possible.

While you can run cron tasks in serverless applications by using scheduled expressions for rules in Amazon EventBridge, these should be used sparingly or as a last-resort. In any scheduled task that processes a batch, there is the potential for the volume of transactions to grow beyond what can be processed within the 15-minute Lambda duration limit. If the limitations of external systems force you to use a scheduler, you should generally schedule for the shortest reasonable recurring time period.

For example, it's not best practice to use a batch process that triggers a Lambda function to fetch a list of new Amazon S3 objects. This is because the service may receive more new objects in between batches than can be processed within a 15-minute Lambda function.



Instead, Amazon S3 should invoke the Lambda function each time a new object is put into the bucket. This approach is significantly more scalable and works in near-real time.



Choose an orchestration option for complex workflows

Workflows that involve branching logic, different types of failure models, and retry logic typically use an orchestrator to keep track of the state of the overall execution. Don't build ad-hoc orchestration in standard Lambda functions. This results in tight coupling, complex routing code, and no automatic state recovery.

Instead, use one of these purpose-built orchestration options:

- **Lambda durable functions:** Application-centric orchestration using standard programming languages with automatic checkpointing, built-in retry, and execution recovery. Ideal for developers who prefer keeping workflow logic in code alongside business logic within Lambda.
- **AWS Step Functions:** Visual workflow orchestration with native integrations to 220+ AWS services. Ideal for multi-service coordination, zero-maintenance infrastructure, and visual workflow design.

For guidance on choosing between these options, see Durable functions or Step Functions.

With Step Functions, you use state machines to manage orchestration. This extracts the error handling, routing, and branching logic from your code, replacing it with state machines declared using JSON. Apart from making workflows more robust and observable, you can also add versioning to workflows and make the state machine a codified resource that you can add to a code repository.

It's common for simpler workflows in Lambda functions to become more complex over time. When operating a production serverless application, it's important to identify when this is happening, so you can migrate this logic to a state machine or durable function.

Implement idempotency

AWS serverless services, including Lambda, are fault-tolerant and designed to handle failures. For example, if a service invokes a Lambda function and there is a service disruption, Lambda invokes your function in a different Availability Zone. If your function throws an error, Lambda retries the invocation.

Since the same event may be received more than once, functions should be designed to be idempotent. This means that receiving the same event multiple times does not change the result beyond the first time the event was received.

You can implement idempotency in Lambda functions by using a DynamoDB table to track recently processed identifiers to determine if the transaction has already been handled previously. The DynamoDB table usually implements a Time To Live (TTL) value to expire items to limit the storage space used.

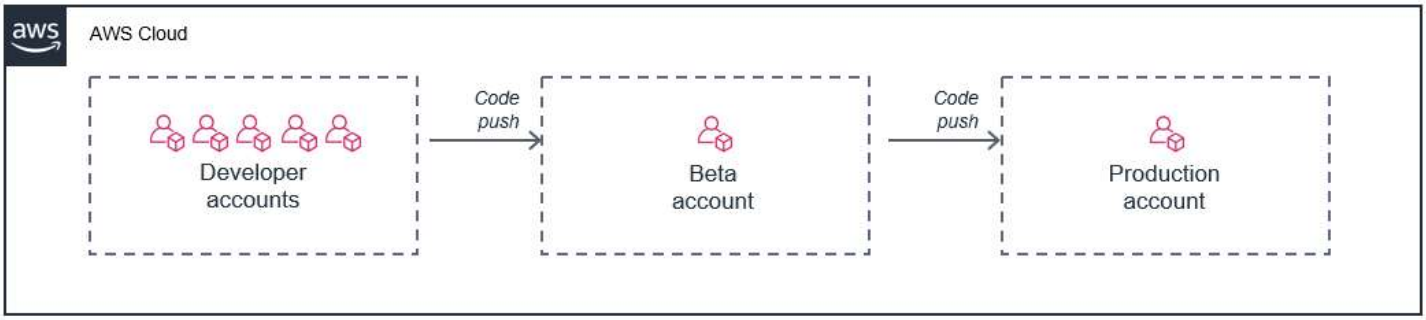
Use multiple AWS accounts for managing quotas

Many service quotas in AWS are set at the account level. This means that as you add more workloads, you can quickly exhaust your limits.

An effective way to solve this issue is to use multiple AWS accounts, dedicating each workload to its own account. This prevents quotas from being shared with other workloads or non-production resources.

In addition, by using AWS Organizations, you can centrally manage the billing, compliance, and security of these accounts. You can attach policies to groups of accounts to avoid custom scripts and manual processes.

One common approach is to provide each developer with an AWS account, and then use separate accounts for a beta deployment stage and production:



In this model, each developer has their own set of limits for the account, so their usage does not impact your production environment. This approach also allows developers to test Lambda functions locally on their development machines against live cloud resources in their individual accounts.

Create your first Lambda function

To get started with Lambda, use the Lambda console to create a function. In a few minutes, you can create and deploy a function and test it in the console.

As you carry out the tutorial, you'll learn some fundamental Lambda concepts, like how to pass arguments to your function using the Lambda *event object*. You'll also learn how to return log outputs from your function, and how to view your function's invocation logs in Amazon CloudWatch Logs.

To keep things simple, you create your function using either the Python or Node.js runtime. With these interpreted languages, you can edit function code directly in the console's built-in code editor. With compiled languages like Java and C#, you must create a deployment package on your local build machine and upload it to Lambda. To learn about deploying functions to Lambda using other runtimes, see the links in the the section called “Next steps” section.

> **Tip**
>
> To learn how to build **serverless solutions**, check out the Serverless Developer Guide.

Prerequisites

Sign up for an AWS account

If you do not have an AWS account, complete the following steps to create one.

To sign up for an AWS account

1. Open https://portal.aws.amazon.com/billing/signup.
2. Follow the online instructions.

   Part of the sign-up procedure involves receiving a phone call or text message and entering a verification code on the phone keypad.

   When you sign up for an AWS account, an *AWS account root user* is created. The root user has access to all AWS services and resources in the account. As a security best practice, assign administrative access to a user, and use only the root user to perform tasks that require root user access.

AWS sends you a confirmation email after the sign-up process is complete. At any time, you can view your current account activity and manage your account by going to https://aws.amazon.com/ and choosing **My Account**.

Create a user with administrative access

After you sign up for an AWS account, secure your AWS account root user, enable AWS IAM Identity Center, and create an administrative user so that you don't use the root user for everyday tasks.

Secure your AWS account root user

1. Sign in to the AWS Management Console as the account owner by choosing **Root user** and entering your AWS account email address. On the next page, enter your password.

   For help signing in by using root user, see Signing in as the root user in the *AWS Sign-In User Guide*.

2. Turn on multi-factor authentication (MFA) for your root user.

   For instructions, see Enable a virtual MFA device for your AWS account root user (console) in the *IAM User Guide*.

Create a user with administrative access

1. Enable IAM Identity Center.

   For instructions, see Enabling AWS IAM Identity Center in the *AWS IAM Identity Center User Guide*.

2. In IAM Identity Center, grant administrative access to a user.

   For a tutorial about using the IAM Identity Center directory as your identity source, see Configure user access with the default IAM Identity Center directory in the *AWS IAM Identity Center User Guide*.

Sign in as the user with administrative access

- To sign in with your IAM Identity Center user, use the sign-in URL that was sent to your email address when you created the IAM Identity Center user.

For help signing in using an IAM Identity Center user, see Signing in to the AWS access portal in the *AWS Sign-In User Guide*.

Assign access to additional users

1. In IAM Identity Center, create a permission set that follows the best practice of applying least-privilege permissions.

   For instructions, see Create a permission set in the *AWS IAM Identity Center User Guide*.
2. Assign users to a group, and then assign single sign-on access to the group.

   For instructions, see Add groups in the *AWS IAM Identity Center User Guide*.

Create a Lambda function with the console

In this example, your function takes a JSON object containing two integer values labeled `"length"` and `"width"`. The function multiplies these values to calculate an area and returns this as a JSON string.

Your function also prints the calculated area, along with the name of its CloudWatch log group. Later in the tutorial, you'll learn to use CloudWatch Logs to view records of your functions' invocation.

To create a Hello world Lambda function with the console

1. Open the Functions page of the Lambda console.
2. Choose **Create function**.
3. Select **Author from scratch**.
4. In the **Basic information** pane, for **Function name**, enter **`myLambdaFunction`**.
5. For **Runtime**, choose either **Node.js 24** or **Python 3.14**.
6. Leave **architecture** set to **x86_64**, and then choose **Create function**.

In addition to a simple function that returns the message `Hello from Lambda!`, Lambda also creates an execution role for your function. An execution role is an AWS Identity and Access Management (IAM) role that grants a Lambda function permission to access AWS services and

resources. For your function, the role that Lambda creates grants basic permissions to write to CloudWatch Logs.

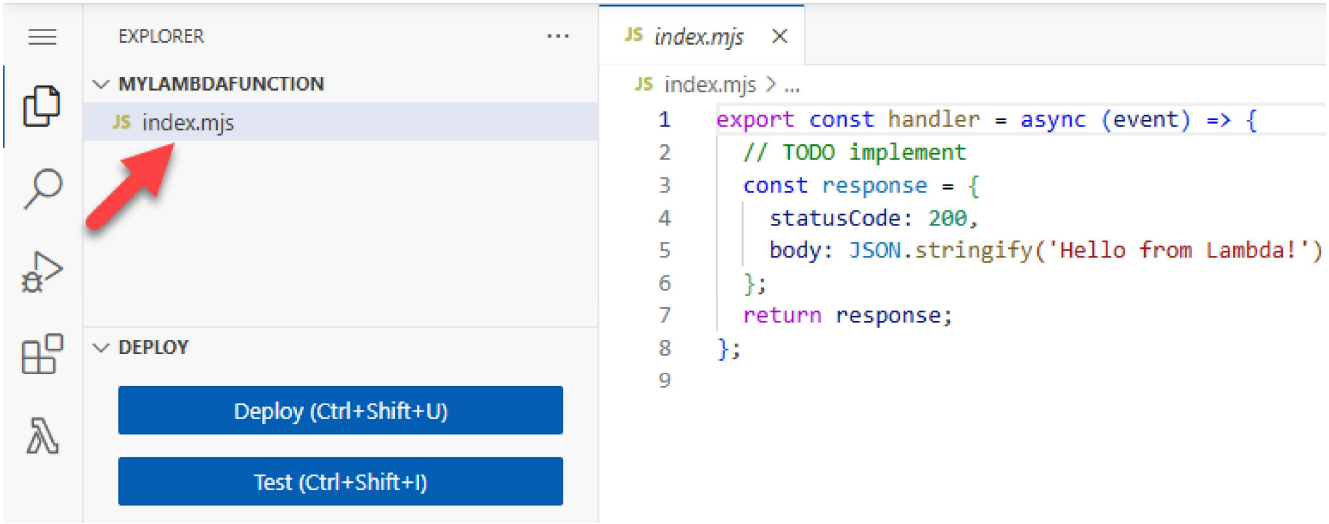
Use the console's built-in code editor to replace the Hello world code that Lambda created with your own function code.

Node.js

To modify the code in the console

1. Choose the **Code** tab.

   In the console's built-in code editor, you should see the function code that Lambda created. If you don't see the **index.mjs** tab in the code editor, select **index.mjs** in the file explorer as shown on the following diagram.

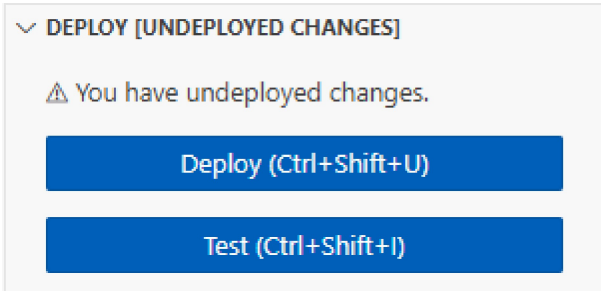


2. Paste the following code into the **index.mjs** tab, replacing the code that Lambda created.

```
export const handler = async (event, context) => {

  const length = event.length;
  const width = event.width;
  let area = calculateArea(length, width);
  console.log(`The area is ${area}`);

  console.log('CloudWatch log group: ', context.logGroupName);

  let data = {
    "area": area,
  };
```

```
    return JSON.stringify(data);

  function calculateArea(length, width) {
    return length * width;
  }
};
```

3. In the **DEPLOY** section, choose **Deploy** to update your function's code:



Understanding your function code

Before you move to the next step, let's take a moment to look at the function code and understand some key Lambda concepts.

- The Lambda handler:

  Your Lambda function contains a Node.js function named `handler`. A Lambda function in Node.js can contain more than one Node.js function, but the *handler* function is always the entry point to your code. When your function is invoked, Lambda runs this method.

  When you created your Hello world function using the console, Lambda automatically set the name of the handler method for your function to `handler`. Be sure not to edit the name of this Node.js function. If you do, Lambda won't be able to run your code when you invoke your function.

  To learn more about the Lambda handler in Node.js, see the section called "Handler".

- The Lambda event object:

  The function `handler` takes two arguments, `event` and `context`. An *event* in Lambda is a JSON formatted document that contains data for your function to process.

  If your function is invoked by another AWS service, the event object contains information about the event that caused the invocation. For example, if your function is invoked when

an object is uploaded to an Amazon Simple Storage Service (Amazon S3) bucket, the event contains the name of the bucket and the object key.

In this example, you'll create an event in the console by entering a JSON formatted document with two key-value pairs.

- The Lambda context object:

  The second argument that your function takes is `context`. Lambda passes the *context object* to your function automatically. The context object contains information about the function invocation and execution environment.

  You can use the context object to output information about your function's invocation for monitoring purposes. In this example, your function uses the `logGroupName` parameter to output the name of its CloudWatch log group.

  To learn more about the Lambda context object in Node.js, see the section called "Context".

- Logging in Lambda:

  With Node.js, you can use console methods like `console.log` and `console.error` to send information to your function's log. The example code uses `console.log` statements to output the calculated area and the name of the function's CloudWatch Logs group. You can also use any logging library that writes to `stdout` or `stderr`.

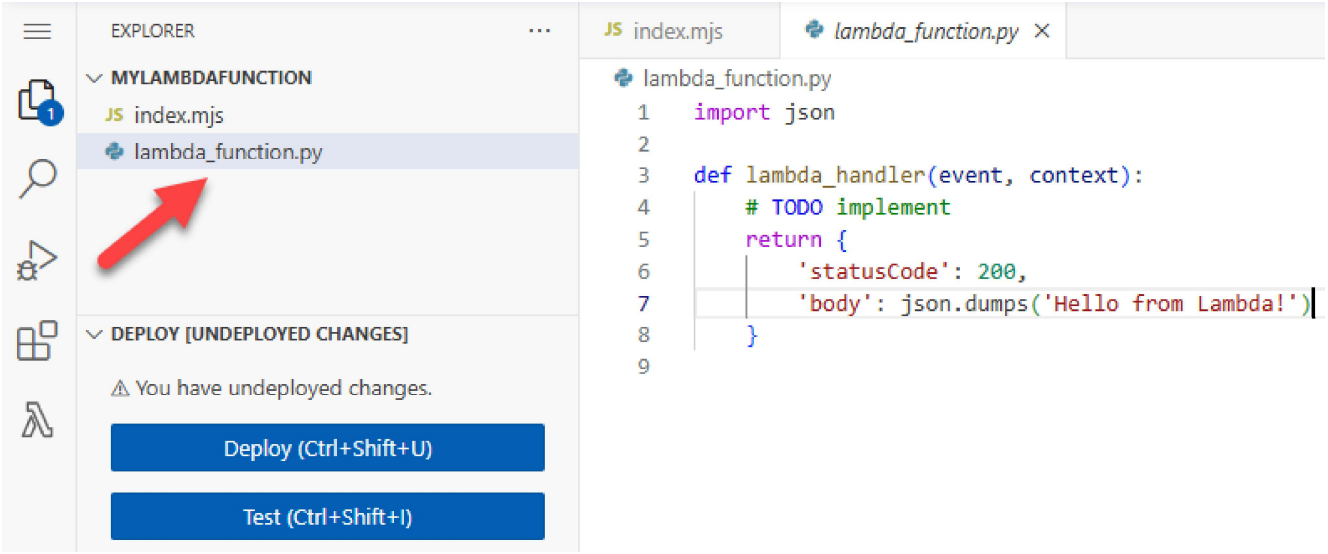
  To learn more, see the section called "Logging". To learn about logging in other runtimes, see the 'Building with' pages for the runtimes you're interested in.

Python

To modify the code in the console

1. Choose the **Code** tab.

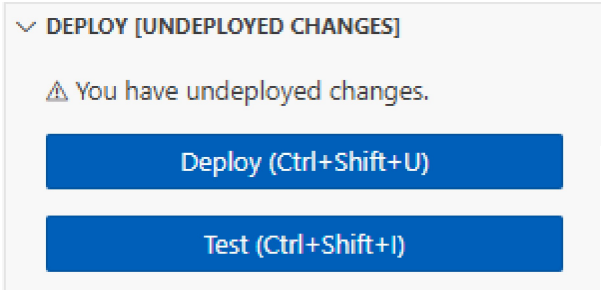
   In the console's built-in code editor, you should see the function code that Lambda created. If you don't see the **lambda_function.py** tab in the code editor, select **lambda_function.py** in the file explorer as shown on the following diagram.



2. Paste the following code into the **lambda_function.py** tab, replacing the code that Lambda created.

```
import json
import logging

logger = logging.getLogger()
logger.setLevel(logging.INFO)

def lambda_handler(event, context):

    # Get the length and width parameters from the event object. The
    # runtime converts the event object to a Python dictionary
    length = event['length']
    width = event['width']

    area = calculate_area(length, width)
    print(f"The area is {area}")

    logger.info(f"CloudWatch logs group: {context.log_group_name}")

    # return the calculated area as a JSON string
    data = {"area": area}
    return json.dumps(data)

def calculate_area(length, width):
    return length*width
```

3. In the **DEPLOY** section, choose **Deploy** to update your function's code:



Understanding your function code

Before you move to the next step, let's take a moment to look at the function code and understand some key Lambda concepts.

- The Lambda handler:

  Your Lambda function contains a Python function named `lambda_handler`. A Lambda function in Python can contain more than one Python function, but the *handler* function is always the entry point to your code. When your function is invoked, Lambda runs this method.

  When you created your Hello world function using the console, Lambda automatically set the name of the handler method for your function to `lambda_handler`. Be sure not to edit the name of this Python function. If you do, Lambda won't be able to run your code when you invoke your function.

  To learn more about the Lambda handler in Python, see the section called "Handler".

- The Lambda event object:

  The function `lambda_handler` takes two arguments, `event` and `context`. An *event* in Lambda is a JSON formatted document that contains data for your function to process.

  If your function is invoked by another AWS service, the event object contains information about the event that caused the invocation. For example, if your function is invoked when an object is uploaded to an Amazon Simple Storage Service (Amazon S3) bucket, the event contains the name of the bucket and the object key.

  In this example, you'll create an event in the console by entering a JSON formatted document with two key-value pairs.

- The Lambda context object:

The second argument that your function takes is `context`. Lambda passes the *context object* to your function automatically. The context object contains information about the function invocation and execution environment.

You can use the context object to output information about your function's invocation for monitoring purposes. In this example, your function uses the `log_group_name` parameter to output the name of its CloudWatch log group.

To learn more about the Lambda context object in Python, see the section called “Context”.

- Logging in Lambda:

  With Python, you can use either a `print` statement or a Python logging library to send information to your function's log. To illustrate the difference in what's captured, the example code uses both methods. In a production application, we recommend that you use a logging library.

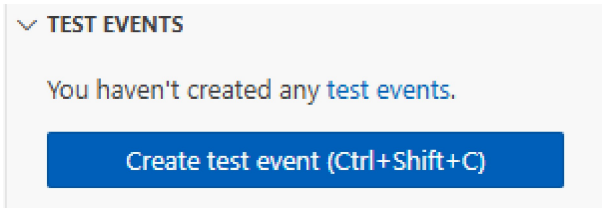
  To learn more, see the section called “Logging”. To learn about logging in other runtimes, see the 'Building with' pages for the runtimes you're interested in.

Invoke the Lambda function using the console code editor

To invoke your function using the Lambda console code editor, create a test event to send to your function. The event is a JSON formatted document containing two key-value pairs with the keys `"length"` and `"width"`.

To create the test event

1. In the **TEST EVENTS** section of the console code editor, choose **Create test event**.



2. For **Event Name**, enter **`myTestEvent`**.
3. In the **Event JSON** section, replace the default JSON with the following:

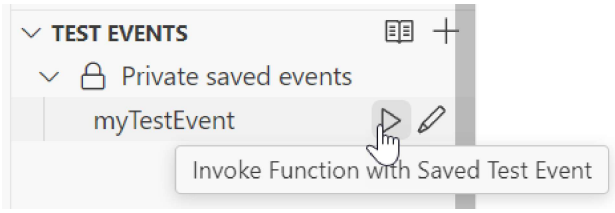
```
{
```

```
  "length": 6,
  "width": 7
}
```

4. Choose **Save**.

To test your function and view invocation records

In the **TEST EVENTS** section of the console code editor, choose the run icon next to your test event:



When your function finishes running, the response and function logs are displayed in the **OUTPUT** tab. You should see results similar to the following:

Node.js

```
Status: Succeeded
Test Event Name: myTestEvent

Response
"{\"area\":42}"

Function Logs
START RequestId: 5c012b0a-18f7-4805-b2f6-40912935034a Version: $LATEST
2024-08-31T23:39:45.313Z 5c012b0a-18f7-4805-b2f6-40912935034a INFO The area is 42
2024-08-31T23:39:45.331Z 5c012b0a-18f7-4805-b2f6-40912935034a INFO CloudWatch log
 group:  /aws/lambda/myLambdaFunction
END RequestId: 5c012b0a-18f7-4805-b2f6-40912935034a
REPORT RequestId: 5c012b0a-18f7-4805-b2f6-40912935034a Duration: 20.67 ms Billed
 Duration: 21 ms Memory Size: 128 MB Max Memory Used: 66 MB Init Duration: 163.87 ms

Request ID
5c012b0a-18f7-4805-b2f6-40912935034a
```

Python

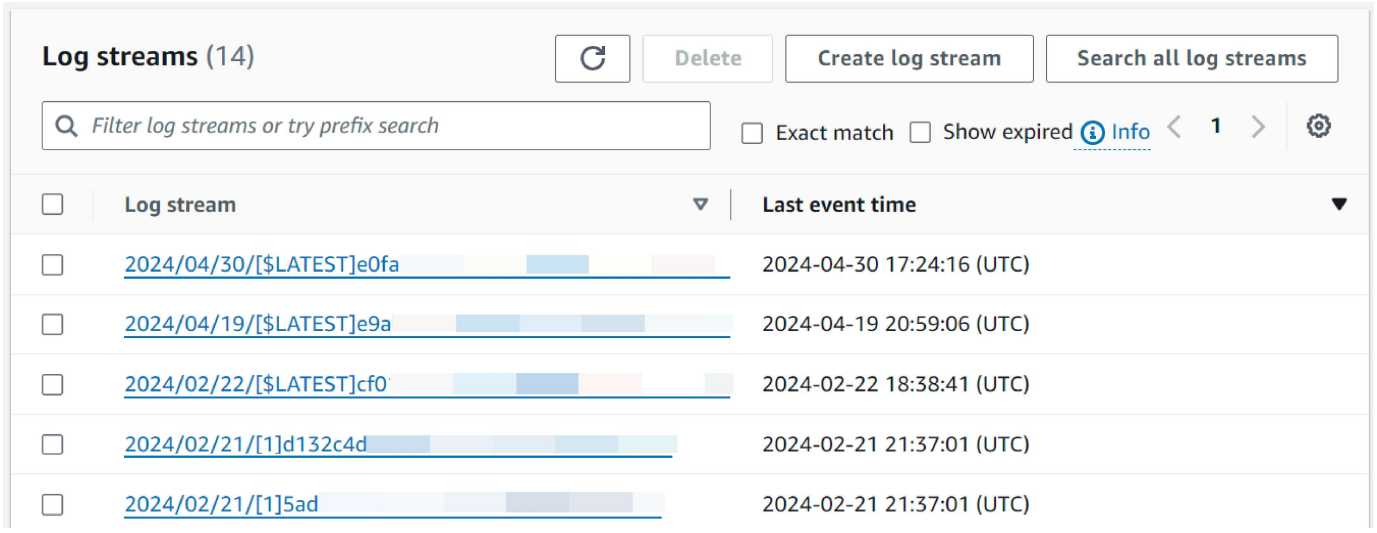
```
Status: Succeeded
Test Event Name: myTestEvent
```

```
Response
"{\"area\": 42}"

Function Logs
START RequestId: 2d0b1579-46fb-4bf7-a6e1-8e08840eae5b Version: $LATEST
The area is 42
[INFO] 2024-08-31T23:43:26.428Z 2d0b1579-46fb-4bf7-a6e1-8e08840eae5b CloudWatch logs
 group: /aws/lambda/myLambdaFunction
END RequestId: 2d0b1579-46fb-4bf7-a6e1-8e08840eae5b
REPORT RequestId: 2d0b1579-46fb-4bf7-a6e1-8e08840eae5b Duration: 1.42 ms Billed
 Duration: 2 ms Memory Size: 128 MB Max Memory Used: 39 MB Init Duration: 123.74 ms

Request ID
2d0b1579-46fb-4bf7-a6e1-8e08840eae5b
```

When you invoke your function outside of the Lambda console, you must use CloudWatch Logs to view your function's execution results.

To view your function's invocation records in CloudWatch Logs

1. Open the Log groups page of the CloudWatch console.
2. Choose the log group for your function (`/aws/lambda/myLambdaFunction`). This is the log group name that your function printed to the console.
3. Scroll down and choose the **Log stream** for the function invocations you want to look at.



You should see output similar to the following:

Node.js

```
INIT_START Runtime Version: nodejs:22.v13    Runtime Version ARN:
 arn:aws:lambda:us-
west-2::runtime:e3aaabf6b92ef8755eaae2f4bfdcb7eb8c4536a5e044900570a42bdba7b869d9
START RequestId: aba6c0fc-cf99-49d7-a77d-26d805dacd20 Version: $LATEST
2024-08-23T22:04:15.809Z    5c012b0a-18f7-4805-b2f6-40912935034a  INFO The area
 is 42
2024-08-23T22:04:15.810Z    aba6c0fc-cf99-49d7-a77d-26d805dacd20  INFO
 CloudWatch log group:  /aws/lambda/myLambdaFunction
END RequestId: aba6c0fc-cf99-49d7-a77d-26d805dacd20
REPORT RequestId: aba6c0fc-cf99-49d7-a77d-26d805dacd20    Duration: 17.77 ms
 Billed Duration: 18 ms    Memory Size: 128 MB    Max Memory Used: 67 MB    Init
 Duration: 178.85 ms
```

Python

```
INIT_START Runtime Version: python:3.13.v16    Runtime Version ARN:
 arn:aws:lambda:us-
west-2::runtime:ca202755c87b9ec2b58856efb7374b4f7b655a0ea3deb1d5acc9aee9e297b072
START RequestId: 9d4096ee-acb3-4c25-be10-8a210f0a9d8e Version: $LATEST
The area is 42
[INFO] 2024-09-01T00:05:22.464Z 9315ab6b-354a-486e-884a-2fb2972b7d84 CloudWatch
 logs group: /aws/lambda/myLambdaFunction
END RequestId: 9d4096ee-acb3-4c25-be10-8a210f0a9d8e
REPORT RequestId: 9d4096ee-acb3-4c25-be10-8a210f0a9d8e    Duration: 1.15 ms
 Billed Duration: 2 ms    Memory Size: 128 MB    Max Memory Used: 40 MB
```

Clean up

When you're finished working with the example function, delete it. You can also delete the log group that stores the function's logs, and the execution role that the console created.

To delete the Lambda function

1. Open the Functions page of the Lambda console.
2. Select the function that you created.
3. Choose **Actions**, **Delete**.
4. Type **`confirm`** in the text input field and choose **Delete**.

To delete the log group

1. Open the Log groups page of the CloudWatch console.
2. Select the function's log group (`/aws/lambda/myLambdaFunction`).
3. Choose **Actions**, **Delete log group(s)**.
4. In the **Delete log group(s)** dialog box, choose **Delete**.

To delete the execution role

1. Open the Roles page of the AWS Identity and Access Management (IAM) console.
2. Select the function's execution role (for example, `myLambdaFunction-role-31exxmpl`).
3. Choose **Delete**.
4. In the **Delete role** dialog box, enter the role name, and then choose **Delete**.

Additional resources and next steps

Now that you've created and tested a simple Lambda function using the console, take these next steps:

- Learn to add dependencies to your function and deploy it using a .zip deployment package. Choose your preferred language from the following links.

  Node.js

  the section called "Deploy .zip file archives"

  Typescript

  the section called "Deploy .zip file archives"

  Python

  the section called "Deploy .zip file archives"

  Ruby

  the section called "Deploy .zip file archives"

  Java

  the section called "Deploy .zip file archives"

Go

the section called “Deploy .zip file archives”

C#

the section called “Deployment package”

- To learn how to invoke a Lambda function using another AWS service, see Tutorial: Using an Amazon S3 trigger to invoke a Lambda function.
- Choose one of the following tutorials for more complex examples of using Lambda with other AWS services.
  - Tutorial: Using Lambda with API Gateway: Create an Amazon API Gateway REST API that invokes a Lambda function.
  - Using a Lambda function to access an Amazon RDS database: Use a Lambda function to write data to an Amazon Relational Database Service (Amazon RDS) database through RDS Proxy.
  - Using an Amazon S3 trigger to create thumbnail images: Use a Lambda function to create a thumbnail every time an image file is uploaded to an Amazon S3 bucket.

Getting started with example applications and patterns

The following resources can be used to quickly create and deploy serverless apps that implement some common Lambda uses cases. For each of the example apps, we provide instructions to either create and configure resources manually using the AWS Management Console, or to use the AWS Serverless Application Model to deploy the resources using IaC. Follow the console intructions to learn more about configuring the individual AWS resources for each app, or use to AWS SAM to quickly deploy resources as you would in a production environment.

File Processing

- **PDF Encryption Application**: Create a serverless application that encrypts PDF files when they are uploaded to an Amazon Simple Storage Service bucket and saves them to another bucket, which is useful for securing sensitive documents upon upload.

- **Image Analysis Application**: Create a serverless application that extracts text from images using Amazon Rekognition, which is useful for document processing, content moderation, and automated image analysis.

Database Integration

- **Queue-to-Database Application**: Create a serverless application that writes queue messages to an Amazon RDS database, which is useful for processing user registrations and handling order submissions.

- **Database Event Handler**: Create a serverless application that responds to Amazon DynamoDB table changes, which is useful for audit logging, data replication, and automated workflows.

Scheduled Tasks

- **Database Maintenance Application**: Create a serverless application that automatically deletes entries more than 12 months old from an Amazon DynamoDB table using a cron schedule, which is useful for automated database maintenance and data lifecycle management.

- **Create an EventBridge scheduled rule for Lambda functions**: Use scheduled expressions for rules in EventBridge to trigger a Lambda function on a timed schedule. This format uses cron syntax and can be set with a one-minute granularity.

Long-running Workflows

- **Order Processing Application**: Create a serverless application using Durable Functions that handles complex order fulfillment, including payment processing, inventory checks, and shipping coordination. This example demonstrates how to build workflows that can run for extended periods while maintaining state.

Additional resources

Use the following resources to further explore Lambda and serverless application development:

- **Serverless Land**: a library of ready-to-use patterns for building serverless apps. It helps developers create applications faster using AWS services like Lambda, API Gateway, and EventBridge. The site offers pre-built solutions and best practices, making it easier to develop serverless systems.
- **Lambda sample applications**: Applications that are available in the GitHub repository for this guide. These samples demonstrate the use of various languages and AWS services. Each sample application includes scripts for easy deployment and cleanup and supporting resources.
- **Code examples for Lambda using AWS SDKs**: Examples that show you how to use Lambda with AWS software development kits (SDKs). These examples include basics, actions, scenarios, and AWS community contributions. Examples cover essential operations, individual service functions, and specific tasks using multiple functions or AWS services.

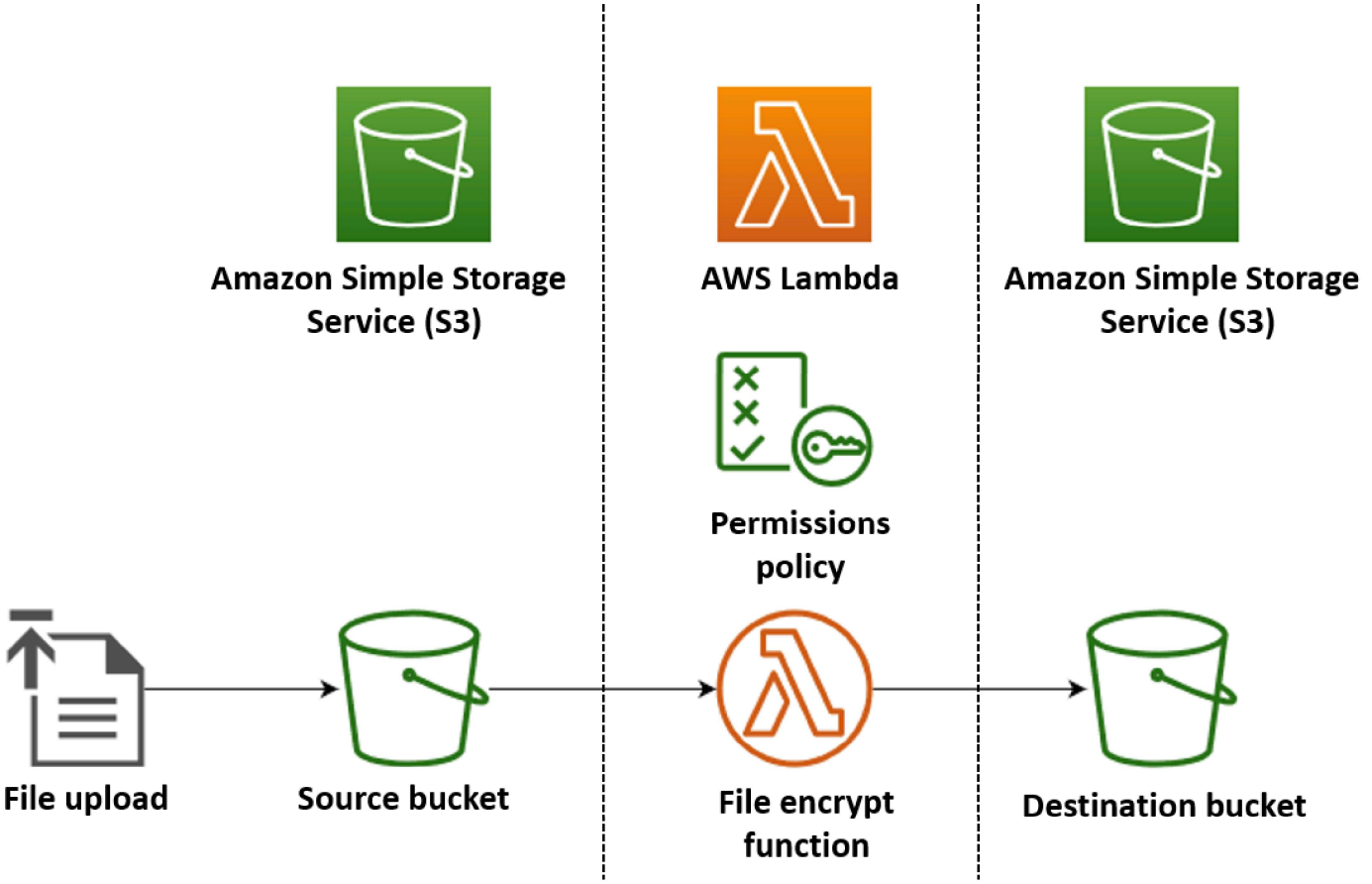
Create a serverless file-processing app

One of the most common use cases for Lambda is to perform file processing tasks. For example, you might use a Lambda function to automatically create PDF files from HTML files or images, or to create thumbnails when a user uploads an image.

In this example, you create an app which automatically encrypts PDF files when they are uploaded to an Amazon Simple Storage Service (Amazon S3) bucket. To implement this app, you create the following resources:

- An S3 bucket for users to upload PDF files to
- A Lambda function in Python which reads the uploaded file and creates an encrypted, password-protected version of it
- A second S3 bucket for Lambda to save the encrypted file in

You also create an AWS Identity and Access Management (IAM) policy to give your Lambda function permission to perform read and write operations on your S3 buckets.



> **Tip**
>
> If you're brand new to Lambda, we recommend that you start with the tutorial *Create your first function* before creating this example app.

You can deploy your app manually by creating and configuring resources with the AWS Management Console or the AWS Command Line Interface (AWS CLI). You can also deploy the app by using the AWS Serverless Application Model (AWS SAM). AWS SAM is an infrastructure as code

(IaC) tool. With IaC, you don't create resources manually, but define them in code and then deploy them automatically.

If you want to learn more about using Lambda with IaC before deploying this example app, see the section called "Infrastructure as code (IaC)".

Create the Lambda function source code files

Create the following files in your project directory:

- `lambda_function.py` - the Python function code for the Lambda function that performs the file encryption
- `requirements.txt` - a manifest file defining the dependencies that your Python function code requires

Expand the following sections to view the code and to learn more about the role of each file. To create the files on your local machine, either copy and paste the code below, or download the files from the aws-lambda-developer-guide GitHub repo.

Python function code

Copy and paste the following code into a file named `lambda_function.py`.

```
from pypdf import PdfReader, PdfWriter
import uuid
import os
from urllib.parse import unquote_plus
import boto3

# Create the S3 client to download and upload objects from S3
s3_client = boto3.client('s3')

def lambda_handler(event, context):
    # Iterate over the S3 event object and get the key for all uploaded files
    for record in event['Records']:
        bucket = record['s3']['bucket']['name']
        key = unquote_plus(record['s3']['object']['key']) # Decode the S3 object key to
 remove any URL-encoded characters
        download_path = f'/tmp/{uuid.uuid4()}.pdf' # Create a path in the Lambda tmp
 directory to save the file to
```

```
        upload_path = f'/tmp/converted-{uuid.uuid4()}.pdf' # Create another path to
 save the encrypted file to

        # If the file is a PDF, encrypt it and upload it to the destination S3 bucket
        if key.lower().endswith('.pdf'):
            s3_client.download_file(bucket, key, download_path)
            encrypt_pdf(download_path, upload_path)
            encrypted_key = add_encrypted_suffix(key)
            s3_client.upload_file(upload_path, f'{bucket}-encrypted', encrypted_key)

# Define the function to encrypt the PDF file with a password
def encrypt_pdf(file_path, encrypted_file_path):
    reader = PdfReader(file_path)
    writer = PdfWriter()

    for page in reader.pages:
        writer.add_page(page)

    # Add a password to the new PDF
    writer.encrypt("my-secret-password")

    # Save the new PDF to a file
    with open(encrypted_file_path, "wb") as file:
        writer.write(file)

# Define a function to add a suffix to the original filename after encryption
def add_encrypted_suffix(original_key):
    filename, extension = original_key.rsplit('.', 1)
    return f'{filename}_encrypted.{extension}'
```

Note

In this example code, a password for the encrypted file (`my-secret-password`) is hardcoded into the function code. In a production application, don't include sensitive information like passwords in your function code. Instead, create an AWS Secrets Manager secret and then use the AWS Parameters and Secrets Lambda extension to retrieve your credentials in your Lambda function.

The python function code contains three functions - the handler function that Lambda runs when your function is invoked, and two separate function named `add_encrypted_suffix` and `encrypt_pdf` that the handler calls to perform the PDF encryption.

When your function is invoked by Amazon S3, Lambda passes a JSON formatted *event* argument to the function that contains details about the event that caused the invocation. In this case, the information includes name of the S3 bucket and the object keys for the uploaded files. To learn more about the format of event object for Amazon S3, see the section called "S3".

Your function then uses the AWS SDK for Python (Boto3) to download the PDF files specified in the event object to its local temporary storage directory, before encrypting them using the `pypdf` library.

Finally, the function uses the Boto3 SDK to store the encrypted file in your S3 destination bucket.

`requirements.txt` manifest file

Copy and paste the following code into a file named `requirements.txt`.

```
boto3
pypdf
```

For this example, your function code has only two dependencies that aren't part of the standard Python library - the SDK for Python (Boto3) and the `pypdf` package the function uses to perform the PDF encryption.

> **Note**
>
> A version of the SDK for Python (Boto3) is included as part of the Lambda runtime, so your code would run without adding Boto3 to your function's deployment package. However, to maintain full control of your function's dependencies and avoid possible issues with version misalignment, best practice for Python is to include all function dependencies in your function's deployment package. See the section called "Runtime dependencies in Python" to learn more.

Deploy the app

You can create and deploy the resources for this example app either manually or by using AWS SAM. In a production environment, we recommend that you use an IaC tool like AWS SAM to quickly and repeatably deploy whole serverless applications without using manual processes.

Deploy the resources manually

To deploy your app manually:

- Create source and destination Amazon S3 buckets
- Create a Lambda function that encrypts a PDF file and saves the encrypted version to an S3 bucket
- Configure a Lambda trigger that invokes your function when objects are uploaded to your source bucket

Before you begin, make sure that Python is installed on your build machine.

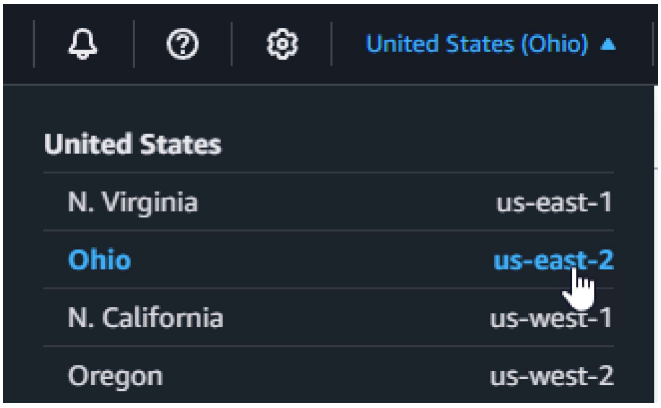
Create two S3 buckets

First create two S3 buckets. The first bucket is the source bucket you will upload your PDF files to. The second bucket is used by Lambda to save the encrypted file when you invoke your function.

Console

To create the S3 buckets (console)

1. Open the General purpose buckets page of the Amazon S3 console.
2. Select the AWS Region closest to your geographical location. You can change your region using the drop-down list at the top of the screen.



3. Choose **Create bucket**.
4. Under **General configuration**, do the following:
   a. For **Bucket type**, ensure **General purpose** is selected.
   b. For **Bucket name**, enter a globally unique name that meets the Amazon S3 bucket naming rules. Bucket names can contain only lower case letters, numbers, dots (.), and hyphens (-).
5. Leave all other options set to their default values and choose **Create bucket**.
6. Repeat steps 1 to 4 to create your destination bucket. For **Bucket name**, enter **`amzn-s3-demo-bucket-encrypted`**, where **`amzn-s3-demo-bucket`** is the name of the source bucket you just created.

AWS CLI

Before you begin, make sure that the AWS CLI is installed on your build machine.

To create the Amazon S3 buckets (AWS CLI)

1. Run the following CLI command to create your source bucket. The name you choose for your bucket must be globally unique and follow the Amazon S3 bucket naming rules. Names can only contain lower case letters, numbers, dots (.), and hyphens (-). For `region` and `LocationConstraint`, choose the AWS Region closest to your geographical location.

```
aws s3api create-bucket --bucket amzn-s3-demo-bucket --region us-east-2 \
--create-bucket-configuration LocationConstraint=us-east-2
```

   Later in the tutorial, you must create your Lambda function in the same AWS Region as your source bucket, so make a note of the region you chose.
2. Run the following command to create your destination bucket. For the bucket name, you must use **`amzn-s3-demo-bucket-encrypted`**, where **`amzn-s3-demo-bucket`** is the name of the source bucket you created in step 1. For `region` and `LocationConstraint`, choose the same AWS Region you used to create your source bucket.

```
aws s3api create-bucket --bucket amzn-s3-demo-bucket-encrypted --region us-
east-2 \
--create-bucket-configuration LocationConstraint=us-east-2
```

Create an execution role

An execution role is an IAM role that grants a Lambda function permission to access AWS services and resources. To give your function read and write access to Amazon S3, you attach the AWS managed policy `AmazonS3FullAccess`.

Console

To create an execution role and attach the `AmazonS3FullAccess` managed policy (console)

1. Open the Roles page in the IAM console.
2. Choose **Create role**.
3. For **Trusted entity type**, select **AWS service**, and for **Use case**, select **Lambda**.
4. Choose **Next**.
5. Add the `AmazonS3FullAccess` managed policy by doing the following:
   a. In **Permissions policies**, enter **`AmazonS3FullAccess`** into the search bar.
   b. Select the checkbox next to the policy.
   c. Choose **Next**.
6. In **Role details**, for **Role name** enter **`LambdaS3Role`**.
7. Choose **Create Role**.

AWS CLI

To create an execution role and attach the `AmazonS3FullAccess` managed policy (AWS CLI)

1. Save the following JSON in a file named `trust-policy.json`. This trust policy allows Lambda to use the role's permissions by giving the service principal `lambda.amazonaws.com` permission to call the AWS Security Token Service (AWS STS) `AssumeRole` action.

```
{
  "Version":"2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
```

```
        "Service": "lambda.amazonaws.com"
      },
      "Action": "sts:AssumeRole"
    }
  ]
}
```

2. From the directory you saved the JSON trust policy document in, run the following CLI command to create the execution role.

```
aws iam create-role --role-name LambdaS3Role --assume-role-policy-document
 file://trust-policy.json
```

3. To attach the `AmazonS3FullAccess` managed policy, run the following CLI command.

```
aws iam attach-role-policy --role-name LambdaS3Role --policy-arn
 arn:aws:iam::aws:policy/AmazonS3FullAccess
```

Create the function deployment package

To create your function, you create a *deployment package* containing your function code and its dependencies. For this application, your function code uses a separate library for the PDF encryption.

To create the deployment package

1. Navigate to the project directory containing the `lambda_function.py` and `requirements.txt` files you created or downloaded from GitHub earlier and create a new directory named `package`.
2. Install the dependencies specified in the `requirements.txt` file in your `package` directory by running the following command.

```
pip install -r requirements.txt --target ./package/
```

3. Create a .zip file containing your application code and its dependencies. In Linux or MacOS, run the following commands from your command line interface.

```
cd package
zip -r ../lambda_function.zip .
cd ..
```

```
zip lambda_function.zip lambda_function.py
```

In Windows, use your preferred zip tool to create the `lambda_function.zip` file. Make sure that your `lambda_function.py` file and the folders containing your dependencies are all at the root of the .zip file.

You can also create your deployment package using a Python virtual environment. See Working with .zip file archives for Python Lambda functions

Create the Lambda function

You now use the deployment package you created in the previous step to deploy your Lambda function.

Console

To create the function (console)

To create your Lambda function using the console, you first create a basic function containing some 'Hello world' code. You then replace this code with your own function code by uploading the.zip file you created in the previous step.

To ensure that your function doesn't time out when encrypting large PDF files, you configure the function's memory and timeout settings. You also set the function's log format to JSON. Configuring JSON formatted logs is necessary when using the provided test script so it can read the function's invocation status from CloudWatch Logs to confirm successful invocation.

1. Open the Functions page of the Lambda console.
2. Make sure you're working in the same AWS Region you created your S3 bucket in. You can change your region using the drop-down list at the top of the screen.

3. Choose **Create function**.
4. Choose **Author from scratch**.
5. Under **Basic information**, do the following:
    a. For **Function name**, enter **`EncryptPDF`**.
    b. For **Runtime** choose **Python 3.12**.
    c. For **Architecture**, choose **x86_64**.
6. Attach the execution role you created in the previous step by doing the following:
    a. Expand the **Change default execution role** section.
    b. Select **Use an existing role**.
    c. Under **Existing role**, select your role (`LambdaS3Role`).
7. Choose **Create function**.

To upload the function code (console)

1. In the **Code source** pane, choose **Upload from**.
2. Choose **.zip file**.
3. Choose **Upload**.
4. In the file selector, select your .zip file and choose **Open**.
5. Choose **Save**.

To configure the function memory and timeout (console)

1. Select the **Configuration** tab for your function.
2. In the **General configuration** pane, choose **Edit**.
3. Set **Memory** to 256 MB and **Timeout** to 15 seconds.
4. Choose **Save**.

To configure the log format (console)

1. Select the **Configuration** tab for your function.
2. Select **Monitoring and operations tools**.
3. In the **Logging configuration** pane, choose **Edit**.

4. For **Logging configuration**, select **JSON**.
5. Choose **Save**.

AWS CLI

To create the function (AWS CLI)

- Run the following command from the directory containing your `lambda_function.zip` file.For the `region` parameter, replace `us-east-2` with the region you created your S3 buckets in.

```
aws lambda create-function --function-name EncryptPDF \
--zip-file fileb://lambda_function.zip --handler lambda_function.lambda_handler
 \
--runtime python3.12 --timeout 15 --memory-size 256 \
--role arn:aws:iam::123456789012:role/LambdaS3Role --region us-east-2 \
--logging-config LogFormat=JSON
```

Configure an Amazon S3 trigger to invoke the function

For your Lambda function to run when you upload a file to your source bucket, you need to configure a trigger for your function. You can configure the Amazon S3 trigger using either the console or the AWS CLI.

> **Important**
>
> This procedure configures the S3 bucket to invoke your function every time that an object is created in the bucket. Be sure to configure this only on the source bucket. If your Lambda function creates objects in the same bucket that invokes it, your function can be invoked continuously in a loop. This can result in un expected charges being billed to your AWS account.

Console

To configure the Amazon S3 trigger (console)

1. Open the Functions page of the Lambda console and choose your function (`EncryptPDF`).

2. Choose **Add trigger**.
3. Select **S3**.
4. Under **Bucket**, select your source bucket.
5. Under **Event types**, select **All object create events**.
6. Under **Recursive invocation**, select the check box to acknowledge that using the same S3 bucket for input and output is not recommended. You can learn more about recursive invocation patterns in Lambda by reading Recursive patterns that cause run-away Lambda functions in Serverless Land.
7. Choose **Add**.

   When you create a trigger using the Lambda console, Lambda automatically creates a resource based policy to give the service you select permission to invoke your function.

AWS CLI

To configure the Amazon S3 trigger (AWS CLI)

1. Add a resource based policy to your function that allows your Amazon S3 source bucket to invoke your function when you add a file. A resource-based policy statement gives other AWS services permission to invoke your function. To give Amazon S3 permission to invoke your function, run the following CLI command. Be sure to replace the `source-account` parameter with your own AWS account ID and to use your own source bucket name.

```
aws lambda add-permission --function-name EncryptPDF \
--principal s3.amazonaws.com --statement-id s3invoke --action
 "lambda:InvokeFunction" \
--source-arn arn:aws:s3:::amzn-s3-demo-bucket \
--source-account 123456789012
```

   The policy you define with this command allows Amazon S3 to invoke your function only when an action takes place on your source bucket.

   > **Note**
   >
   > Although S3 bucket names are globally unique, when using resource-based policies it is best practice to specify that the bucket must belong to your account. This is

because if you delete a bucket, it is possible for another AWS account to create a bucket with the same Amazon Resource Name (ARN).

2. Save the following JSON in a file named `notification.json`. When applied to your source bucket, this JSON configures the bucket to send a notification to your Lambda function every time a new object is added. Replace the AWS account number and AWS Region in the Lambda function ARN with your own account number and region.

```
{
"LambdaFunctionConfigurations": [
    {
      "Id": "EncryptPDFEventConfiguration",
      "LambdaFunctionArn": "arn:aws:lambda:us-
east-2:123456789012:function:EncryptPDF",
      "Events": [ "s3:ObjectCreated:Put" ]
    }
  ]
}
```

3. Run the following CLI command to apply the notification settings in the JSON file you created to your source bucket. Replace `amzn-s3-demo-bucket` with the name of your own source bucket.

```
aws s3api put-bucket-notification-configuration --bucket amzn-s3-demo-bucket \
--notification-configuration file://notification.json
```

To learn more about the `put-bucket-notification-configuration` command and the `notification-configuration` option, see put-bucket-notification-configuration in the *AWS CLI Command Reference*.

Deploy the resources using AWS SAM

Before you begin, make sure that Docker and the latest version of the AWS SAM CLI are installed on your build machine.

1. In your project directory, copy and paste the following code into a file named `template.yaml`. Replace the placeholder bucket names:

- For the source bucket, replace `amzn-s3-demo-bucket` with any name that complies with the S3 bucket naming rules.
- For the destination bucket, replace `amzn-s3-demo-bucket-encrypted` with `<source-bucket-name>-encrypted`, where `<source-bucket>` is the name you chose for your source bucket.

```
AWSTemplateFormatVersion: '2010-09-09'
Transform: AWS::Serverless-2016-10-31

Resources:
  EncryptPDFFunction:
    Type: AWS::Serverless::Function
    Properties:
      FunctionName: EncryptPDF
      Architectures: [x86_64]
      CodeUri: ./
      Handler: lambda_function.lambda_handler
      Runtime: python3.12
      Timeout: 15
      MemorySize: 256
      LoggingConfig:
        LogFormat: JSON
      Policies:
        - AmazonS3FullAccess
      Events:
        S3Event:
          Type: S3
          Properties:
            Bucket: !Ref PDFSourceBucket
            Events: s3:ObjectCreated:*

  PDFSourceBucket:
    Type: AWS::S3::Bucket
    Properties:
      BucketName: amzn-s3-demo-bucket

  EncryptedPDFBucket:
    Type: AWS::S3::Bucket
    Properties:
      BucketName: amzn-s3-demo-bucket-encrypted
```

The AWS SAM template defines the resources you create for your app. In this example, the template defines a Lambda function using the `AWS::Serverless::Function` type and two S3 buckets using the `AWS::S3::Bucket` type. The bucket names specified in the template are placeholders. Before you deploy the app using AWS SAM, you need to edit the template to rename the buckets with globally unique names that meet the S3 bucket naming rules. This step is explained further in the section called "Deploy the resources using AWS SAM".

The definition of the Lambda function resource configures a trigger for the function using the `S3Event` event property. This trigger causes your function to be invoked whenever an object is created in your source bucket.

The function definition also specifies an AWS Identity and Access Management (IAM) policy to be attached to the function's execution role. The AWS managed policy `AmazonS3FullAccess` gives your function the permissions it needs to read and write objects to Amazon S3.

2. Run the following command from the directory in which you saved your `template.yaml`, `lambda_function.py`, and `requirements.txt`files.

```
sam build --use-container
```

This command gathers the build artifacts for your application and places them in the proper format and location to deploy them. Specifying the `--use-container` option builds your function inside a Lambda-like Docker container. We use it here so you don't need to have Python 3.12 installed on your local machine for the build to work.

During the build process, AWS SAM looks for the Lambda function code in the location you specified with the `CodeUri` property in the template. In this case, we specified the current directory as the location (`./`).

If a `requirements.txt` file is present, AWS SAM uses it to gather the specified dependencies. By default, AWS SAM creates a .zip deployment package with your function code and dependencies. You can also choose to deploy your function as a container image using the PackageType property.

3. To deploy your application and create the Lambda and Amazon S3 resources specified in your AWS SAM template, run the following command.

```
sam deploy --guided
```

Using the `--guided` flag means that AWS SAM will show you prompts to guide you through the deployment process. For this deployment, accept the default options by pressing Enter.

During the deployment process, AWS SAM creates the following resources in your AWS account:

- An CloudFormation stack named `sam-app`
- A Lambda function with the name `EncryptPDF`
- Two S3 buckets with the names you chose when you edited the `template.yaml` AWS SAM template file
- An IAM execution role for your function with the name format `sam-app-EncryptPDFFunctionRole-`*`2qGaapHFWOQ8`*

When AWS SAM finishes creating your resources, you should see the following message:

```
Successfully created/updated stack - sam-app in us-east-2
```

Test the app

To test your app, upload a PDF file to your source bucket, and confirm that Lambda creates an encrypted version of the file in your destination bucket. In this example, you can either test this manually using the console or the AWS CLI, or by using the provided test script.

For production applications, you can use traditional test methods and techniques, such as unit testing, to confirm the correct functioning of your Lambda function code. Best practice is also to conduct tests like those in the provided test script which perform integration testing with real, cloud-based resources. Integration testing in the cloud confirms that your infrastructure has been correctly deployed and that events flow between different services as expected. To learn more, see *Testing serverless functions*.

Testing the app manually

You can test your function manually by adding a PDF file to your Amazon S3 source bucket. When you add your file to the source bucket, your Lambda function should be automatically invoked and should store an encrypted version of the file in your target bucket.

Console

To test your app by uploading a file (console)

1. To upload a PDF file to your S3 bucket, do the following:

   a. Open the Buckets page of the Amazon S3 console and choose your source bucket.

   b. Choose **Upload**.

   c. Choose **Add files** and use the file selector to choose the PDF file you want to upload.

   d. Choose **Open**, then choose **Upload**.

2. Verify that Lambda has saved an encrypted version of your PDF file in your target bucket by doing the following:

   a. Navigate back to the Buckets page of the Amazon S3 console and choose your destination bucket.

   b. In the **Objects** pane, you should now see a file with name format `filename_encrypted.pdf` (where `filename.pdf` was the name of the file you uploaded to your source bucket). To download your encrypted PDF, select the file, then choose **Download**.

   c. Confirm that you can open the downloaded file with the password your Lambda function protected it with (`my-secret-password`).

AWS CLI

To test your app by uploading a file (AWS CLI)

1. From the directory containing the PDF file you want to upload, run the following CLI command. Replace the `--bucket` parameter with the name of your source bucket. For the `--key` and `--body` parameters, use the filename of your test file.

```
aws s3api put-object --bucket amzn-s3-demo-bucket --key test.pdf --body ./
test.pdf
```

2. Verify that your function has created an encrypted version of your file and saved it to your target S3 bucket. Run the following CLI command, replacing `amzn-s3-demo-bucket-encrypted` with the name of your own destination bucket.

```
aws s3api list-objects-v2 --bucket amzn-s3-demo-bucket-encrypted
```

If your function runs successfully, you'll see output similar to the following. Your target bucket should contain a file with the name format *<your_test_file>*_encrypted.pdf, where <your_test_file> is the name of the file you uploaded.

```
{
    "Contents": [
        {
            "Key": "test_encrypted.pdf",
            "LastModified": "2023-06-07T00:15:50+00:00",
            "ETag": "\"7781a43e765a8301713f533d70968a1e\"",
            "Size": 2763,
            "StorageClass": "STANDARD"
        }
    ]
}
```

3. To download the file that Lambda saved in your destination bucket, run the following CLI command. Replace the `--bucket` parameter with the name of your destination bucket. For the `--key` parameter, use the filename *<your_test_file>*_encrypted.pdf, where <your_test_file> is the name of the the test file you uploaded.

```
aws s3api get-object --bucket amzn-s3-demo-bucket-encrypted --
key test_encrypted.pdf my_encrypted_file.pdf
```

This command downloads the file to your current directory and saves it as my_encrypted_file.pdf.

4. Confirm the you can open the downloaded file with the password your Lambda function protected it with (`my-secret-password`).

Testing the app with the automated script

Create the following files in your project directory:

- `test_pdf_encrypt.py` - a test script you can use to automatically test your application
- `pytest.ini` - a configuration file for the the test script

Expand the following sections to view the code and to learn more about the role of each file.

Automated test script

Copy and paste the following code into a file named `test_pdf_encrypt.py`. Be sure to replace the placeholder bucket names:

- In the `test_source_bucket_available` function, replace `amzn-s3-demo-bucket` with the name of your source bucket.
- In the `test_encrypted_file_in_bucket` function, replace `amzn-s3-demo-bucket-encrypted` with *`source-bucket`*`-encrypted`, where `source-bucket>` is the name of your source bucket.
- In the `cleanup` function, replace `amzn-s3-demo-bucket` with the name of your source bucket, and replace `amzn-s3-demo-bucket-encrypted` with the name of your destination bucket.

```
import boto3
import json
import pytest
import time
import os

@pytest.fixture
def lambda_client():
    return boto3.client('lambda')

@pytest.fixture
def s3_client():
    return boto3.client('s3')

@pytest.fixture
def logs_client():
    return boto3.client('logs')

@pytest.fixture(scope='session')
def cleanup():
    # Create a new S3 client for cleanup
    s3_client = boto3.client('s3')

    yield
    # Cleanup code will be executed after all tests have finished
```

```
    # Delete test.pdf from the source bucket
    source_bucket = 'amzn-s3-demo-bucket'
    source_file_key = 'test.pdf'
    s3_client.delete_object(Bucket=source_bucket, Key=source_file_key)
    print(f"\nDeleted {source_file_key} from {source_bucket}")

    # Delete test_encrypted.pdf from the destination bucket
    destination_bucket = 'amzn-s3-demo-bucket-encrypted'
    destination_file_key = 'test_encrypted.pdf'
    s3_client.delete_object(Bucket=destination_bucket, Key=destination_file_key)
    print(f"Deleted {destination_file_key} from {destination_bucket}")


@pytest.mark.order(1)
def test_source_bucket_available(s3_client):
    s3_bucket_name = 'amzn-s3-demo-bucket'
    file_name = 'test.pdf'
    file_path = os.path.join(os.path.dirname(__file__), file_name)

    file_uploaded = False
    try:
        s3_client.upload_file(file_path, s3_bucket_name, file_name)
        file_uploaded = True
    except:
        print("Error: couldn't upload file")

    assert file_uploaded, "Could not upload file to S3 bucket"



@pytest.mark.order(2)
def test_lambda_invoked(logs_client):

    # Wait for a few seconds to make sure the logs are available
    time.sleep(5)

    # Get the latest log stream for the specified log group
    log_streams = logs_client.describe_log_streams(
        logGroupName='/aws/lambda/EncryptPDF',
        orderBy='LastEventTime',
        descending=True,
        limit=1
    )
```

```
    latest_log_stream_name = log_streams['logStreams'][0]['logStreamName']

    # Retrieve the log events from the latest log stream
    log_events = logs_client.get_log_events(
        logGroupName='/aws/lambda/EncryptPDF',
        logStreamName=latest_log_stream_name
    )

    success_found = False
    for event in log_events['events']:
        message = json.loads(event['message'])
        status = message.get('record', {}).get('status')
        if status == 'success':
            success_found = True
            break

    assert success_found, "Lambda function execution did not report 'success' status in
 logs."

@pytest.mark.order(3)
def test_encrypted_file_in_bucket(s3_client):
    # Specify the destination S3 bucket and the expected converted file key
    destination_bucket = 'amzn-s3-demo-bucket-encrypted'
    converted_file_key = 'test_encrypted.pdf'

    try:
        # Attempt to retrieve the metadata of the converted file from the destination
 S3 bucket
        s3_client.head_object(Bucket=destination_bucket, Key=converted_file_key)
    except s3_client.exceptions.ClientError as e:
        # If the file is not found, the test will fail
        pytest.fail(f"Converted file '{converted_file_key}' not found in the
 destination bucket: {str(e)}")

def test_cleanup(cleanup):
    # This test uses the cleanup fixture and will be executed last
    pass
```

The automated test script executes three test functions to confirm correct operation of your app:

- The test `test_source_bucket_available` confirms that your source bucket has been successfully created by uploading a test PDF file to the bucket.

- The test `test_lambda_invoked` interrogates the latest CloudWatch Logs log stream for your function to confirm that when you uploaded the test file, your Lambda function ran and reported success.
- The test `test_encrypted_file_in_bucket` confirms that your destination bucket contains the encrypted `test_encrypted.pdf` file.

After all these tests have run, the script runs an additional cleanup step to delete the `test.pdf` and `test_encrypted.pdf` files from both your source and destination buckets.

As with the AWS SAM template, the bucket names specified in this file are placeholders. Before running the test, you need to edit this file with your app's real bucket names. This step is explained further in the section called “Testing the app with the automated script”

Test script configuration file

Copy and paste the following code into a file named `pytest.ini`.

```
[pytest]
markers =
    order: specify test execution order
```

This is needed to specify the order in which the tests in the `test_pdf_encrypt.py` script run.

To run the tests do the following:

1. Ensure that the `pytest` module is installed in your local environment. You can install `pytest` by running the following command:

   ```
   pip install pytest
   ```

2. Save a PDF file named `test.pdf` in the directory containing the `test_pdf_encrypt.py` and `pytest.ini` files.
3. Open a terminal or shell program and run the following command from the directory containing the test files.

   ```
   pytest -s -v
   ```

   When the test completes, you should see output like the following:

```
============================================================= test session starts
 ==============================================================
platform linux -- Python 3.12.2, pytest-7.2.2, pluggy-1.0.0 -- /usr/bin/python3
cachedir: .pytest_cache
hypothesis profile 'default' -> database=DirectoryBasedExampleDatabase('/home/
pdf_encrypt_app/.hypothesis/examples')
Test order randomisation NOT enabled. Enable with --random-order or --random-order-
bucket=<bucket_type>
rootdir: /home/pdf_encrypt_app, configfile: pytest.ini
plugins: anyio-3.7.1, hypothesis-6.70.0, localserver-0.7.1, random-order-1.1.0
collected 4 items

test_pdf_encrypt.py::test_source_bucket_available PASSED
test_pdf_encrypt.py::test_lambda_invoked PASSED
test_pdf_encrypt.py::test_encrypted_file_in_bucket PASSED
test_pdf_encrypt.py::test_cleanup PASSED
Deleted test.pdf from amzn-s3-demo-bucket
Deleted test_encrypted.pdf from amzn-s3-demo-bucket-encrypted


============================================================== 4 passed in 7.32s
 ==============================================================
```

Next steps

Now you've created this example app, you can use the provided code as a basis to create other types of file-processing application. Modify the code in the `lambda_function.py` file to implement the file-processing logic for your use case.

Many typical file-processing use cases involve image processing. When using Python, the most popular image-processing libraries like pillow typically contain C or C++ components. In order to ensure that your function's deployment package is compatible with the Lambda execution environment, it's important to use the correct source distribution binary.

When deploying your resources with AWS SAM, you need to take some extra steps to include the right source distribution in your deployment package. Because AWS SAM won't install dependencies for a different platform than your build machine, specifying the correct source distribution (`.whl` file) in your `requirements.txt` file won't work if your build machine uses an

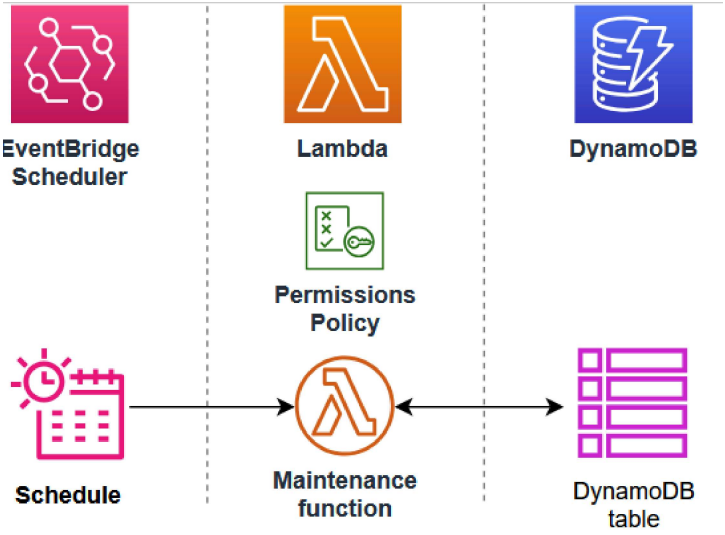
operating system or architecture that's different from the Lambda execution environment. Instead, you should do one of the following:

- Use the `--use-container` option when running `sam build`. When you specify this option, AWS SAM downloads a container base image that's compatible with the Lambda execution environment and builds your function's deployment package in a Docker container using that image. To learn more, see Building a Lambda function inside of a provided container.
- Build your function's .zip deployment package yourself using the correct source distribution binary and save the .zip file in the directory you specify as the `CodeUri` in the AWS SAM template. To learn more about building .zip deployment packages for Python using binary distributions, see the section called “Creating a .zip deployment package with dependencies” and the section called “Creating .zip deployment packages with native libraries”.

Create an app to perform scheduled database maintenance

You can use AWS Lambda to replace scheduled processes such as automated system backups, file conversions, and maintenance tasks. In this example, you create a serverless application that performs regular scheduled maintenance on a DynamoDB table by deleting old entries. The app uses EventBridge Scheduler to invoke a Lambda function on a cron schedule. When invoked, the function queries the table for items older than one year, and deletes them. The function logs each deleted item in CloudWatch Logs.

To implement this example, first create a DynamoDB table and populate it with some test data for your function to query. Then, create a Python Lambda function with an EventBridge Scheduler trigger and an IAM execution role that gives the function permission to read, and delete, items from your table.



> **Tip**
>
> If you're new to Lambda, we recommend that you complete the tutorial *Create your first function* before creating this example app.

You can deploy your app manually by creating and configuring resources with the AWS Management Console. You can also deploy the app by using the AWS Serverless Application Model (AWS SAM). AWS SAM is an infrastructure as code (IaC) tool. With IaC, you don't create resources manually, but define them in code and then deploy them automatically.

If you want to learn more about using Lambda with IaC before deploying this example app, see the section called "Infrastructure as code (IaC)".

Prerequisites

Before you can create the example app, make sure you have the required command line tools and programs installed.

- **Python**

  To populate the DynamoDB table you create to test your app, this example uses a Python script and a CSV file to write data into the table. Make sure you have Python version 3.8 or later installed on your machine.

- **AWS SAM CLI**

If you want to create the DynamoDB table and deploy the example app using AWS SAM, you need to install the AWS SAM CLI. Follow the installation instructions in the *AWS SAM User Guide*.

- **AWS CLI**

  To use the provided Python script to populate your test table, you need to have installed and configured the AWS CLI. This is because the script uses the AWS SDK for Python (Boto3), which needs access to your AWS Identity and Access Management (IAM) credentials. You also need the AWS CLI installed to deploy resources using AWS SAM. Install the CLI by following the installation instructions in the *AWS Command Line Interface User Guide*.

- **Docker**

  To deploy the app using AWS SAM, Docker must also be installed on your build machine. Follow the instructions in Install Docker Engine on the Docker documentation website.

Downloading the example app files

To create the example database and the scheduled-maintenance app, you need to create the following files in your project directory:

Example database files

- `template.yaml` - an AWS SAM template you can use to create the DynamoDB table
- `sample_data.csv` - a CSV file containing sample data to load into your table
- `load_sample_data.py` - a Python script that writes the data in the CSV file into the table

Scheduled-maintenance app files

- `lambda_function.py` - the Python function code for the Lambda function that performs the database maintenance
- `requirements.txt` - a manifest file defining the dependencies that your Python function code requires
- `template.yaml` - an AWS SAM template you can use to deploy the app

Test file

- `test_app.py` - a Python script that scans the table and confirms successful operation of your function by outputting all records older than one year

Expand the following sections to view the code and to learn more about the role of each file in creating and testing your app. To create the files on your local machine, copy and paste the code below.

AWS SAM template (example DynamoDB table)

Copy and paste the following code into a file named `template.yaml`.

```
AWSTemplateFormatVersion: '2010-09-09'
Transform: AWS::Serverless-2016-10-31
Description: SAM Template for DynamoDB Table with Order_number as Partition Key and
 Date as Sort Key

Resources:
  MyDynamoDBTable:
    Type: AWS::DynamoDB::Table
    DeletionPolicy: Retain
    UpdateReplacePolicy: Retain
    Properties:
      TableName: MyOrderTable
      BillingMode: PAY_PER_REQUEST
      AttributeDefinitions:
        - AttributeName: Order_number
          AttributeType: S
        - AttributeName: Date
          AttributeType: S
      KeySchema:
        - AttributeName: Order_number
          KeyType: HASH
        - AttributeName: Date
          KeyType: RANGE
      SSESpecification:
        SSEEnabled: true
      GlobalSecondaryIndexes:
        - IndexName: Date-index
          KeySchema:
            - AttributeName: Date
              KeyType: HASH
          Projection:
```

```
            ProjectionType: ALL
      PointInTimeRecoverySpecification:
        PointInTimeRecoveryEnabled: true

Outputs:
  TableName:
    Description: DynamoDB Table Name
    Value: !Ref MyDynamoDBTable
  TableArn:
    Description: DynamoDB Table ARN
    Value: !GetAtt MyDynamoDBTable.Arn
```

Note

AWS SAM templates use a standard naming convention of `template.yaml`. In this example, you have two template files - one to create the example database and another to create the app itself. Save them in separate sub-directories in your project folder.

This AWS SAM template defines the DynamoDB table resource you create to test your app. The table uses a primary key of `Order_number` with a sort key of `Date`. In order for your Lambda function to find items directly by date, we also define a Global Secondary Index named `Date-index`.

To learn more about creating and configuring a DynamoDB table using the `AWS::DynamoDB::Table` resource, see AWS::DynamoDB::Table in the *AWS CloudFormation User Guide*.

Sample database data file

Copy and paste the following code into a file named `sample_data.csv`.

```
Date,Order_number,CustomerName,ProductID,Quantity,TotalAmount
2023-09-01,ORD001,Alejandro Rosalez,PROD123,2,199.98
2023-09-01,ORD002,Akua Mansa,PROD456,1,49.99
2023-09-02,ORD003,Ana Carolina Silva,PROD789,3,149.97
2023-09-03,ORD004,Arnav Desai,PROD123,1,99.99
2023-10-01,ORD005,Carlos Salazar,PROD456,2,99.98
2023-10-02,ORD006,Diego Ramirez,PROD789,1,49.99
2023-10-03,ORD007,Efua Owusu,PROD123,4,399.96
2023-10-04,ORD008,John Stiles,PROD456,2,99.98
2023-10-05,ORD009,Jorge Souza,PROD789,3,149.97
```

```
2023-10-06,ORD010,Kwaku Mensah,PROD123,1,99.99
2023-11-01,ORD011,Li Juan,PROD456,5,249.95
2023-11-02,ORD012,Marcia Oliveria,PROD789,2,99.98
2023-11-03,ORD013,Maria Garcia,PROD123,3,299.97
2023-11-04,ORD014,Martha Rivera,PROD456,1,49.99
2023-11-05,ORD015,Mary Major,PROD789,4,199.96
2023-12-01,ORD016,Mateo Jackson,PROD123,2,199.99
2023-12-02,ORD017,Nikki Wolf,PROD456,3,149.97
2023-12-03,ORD018,Pat Candella,PROD789,1,49.99
2023-12-04,ORD019,Paulo Santos,PROD123,5,499.95
2023-12-05,ORD020,Richard Roe,PROD456,2,99.98
2024-01-01,ORD021,Saanvi Sarkar,PROD789,3,149.97
2024-01-02,ORD022,Shirley Rodriguez,PROD123,1,99.99
2024-01-03,ORD023,Sofia Martinez,PROD456,4,199.96
2024-01-04,ORD024,Terry Whitlock,PROD789,2,99.98
2024-01-05,ORD025,Wang Xiulan,PROD123,3,299.97
```

This file contains some example test data to populate your DynamoDB table with in a standard comma-separated values (CSV) format.

Python script to load sample data

Copy and paste the following code into a file named `load_sample_data.py`.

```
import boto3
import csv
from decimal import Decimal

# Initialize the DynamoDB client
dynamodb = boto3.resource('dynamodb')
table = dynamodb.Table('MyOrderTable')
print("DDB client initialized.")

def load_data_from_csv(filename):
    with open(filename, 'r') as file:
        csv_reader = csv.DictReader(file)
        for row in csv_reader:
            item = {
                'Order_number': row['Order_number'],
                'Date': row['Date'],
                'CustomerName': row['CustomerName'],
                'ProductID': row['ProductID'],
                'Quantity': int(row['Quantity']),
```

```
                'TotalAmount': Decimal(str(row['TotalAmount']))
            }
            table.put_item(Item=item)
            print(f"Added item: {item['Order_number']} - {item['Date']}")

if __name__ == "__main__":
    load_data_from_csv('sample_data.csv')
    print("Data loading completed.")
```

This Python script first uses the AWS SDK for Python (Boto3) to create a connection to your DynamoDB table. It then iterates over each row in the example-data CSV file, creates an item from that row, and writes the item to the DynamoDB table using the boto3 SDK.

Python function code

Copy and paste the following code into a file named `lambda_function.py`.

```
import boto3
from datetime import datetime, timedelta
from boto3.dynamodb.conditions import Key, Attr
import logging

logger = logging.getLogger()
logger.setLevel("INFO")

def lambda_handler(event, context):
    # Initialize the DynamoDB client
    dynamodb = boto3.resource('dynamodb')

    # Specify the table name
    table_name = 'MyOrderTable'
    table = dynamodb.Table(table_name)

    # Get today's date
    today = datetime.now()

    # Calculate the date one year ago
    one_year_ago = (today - timedelta(days=365)).strftime('%Y-%m-%d')

    # Scan the table using a global secondary index
    response = table.scan(
        IndexName='Date-index',
        FilterExpression='#date < :one_year_ago',
```

```
        ExpressionAttributeNames={
            '#date': 'Date'
        },
        ExpressionAttributeValues={
            ':one_year_ago': one_year_ago
        }
    )

     # Delete old items
    with table.batch_writer() as batch:
        for item in response['Items']:
            Order_number = item['Order_number']
            batch.delete_item(
                Key={
                    'Order_number': Order_number,
                    'Date': item['Date']
                }
            )
            logger.info(f'deleted order number {Order_number}')

    # Check if there are more items to scan
    while 'LastEvaluatedKey' in response:
        response = table.scan(
            IndexName='DateIndex',
            FilterExpression='#date < :one_year_ago',
            ExpressionAttributeNames={
                '#date': 'Date'
            },
            ExpressionAttributeValues={
                ':one_year_ago': one_year_ago
            },
            ExclusiveStartKey=response['LastEvaluatedKey']
        )

        # Delete old items
        with table.batch_writer() as batch:
            for item in response['Items']:
                batch.delete_item(
                    Key={
                        'Order_number': item['Order_number'],
                        'Date': item['Date']
                    }
                )
```

```
    return {
        'statusCode': 200,
        'body': 'Cleanup completed successfully'
    }
```

The Python function code contains the handler function (`lambda_handler`) that Lambda runs when your function is invoked.

When the function is invoked by EventBridge Scheduler, it uses the AWS SDK for Python (Boto3) to create a connection to the DynamoDB table on which the scheduled maintenance task is to be performed. It then uses the Python `datetime` library to calculate the date one year ago, before scanning the table for items older than this and deleting them.

Note that responses from DynamoDB query and scan operations are limited to a maximum of 1 MB in size. If the response is larger than 1 MB, DynamoDB paginates the data and returns a `LastEvaluatedKey` element in the response. To ensure that our function processes all the records in the table, we check for the presence of this key and continue performing table scans from the last evaluated position until the whole table has been scanned.

`requirements.txt` manifest file

Copy and paste the following code into a file named `requirements.txt`.

```
boto3
```

For this example, your function code has only one dependency that isn't part of the standard Python library - the SDK for Python (Boto3) that the function uses to scan and delete items from the DynamoDB table.

> **Note**
>
> A version of the SDK for Python (Boto3) is included as part of the Lambda runtime, so your code would run without adding Boto3 to your function's deployment package. However, to maintain full control of your function's dependencies and avoid possible issues with version misalignment, best practice for Python is to include all function dependencies in your function's deployment package. See the section called “Runtime dependencies in Python” to learn more.

AWS SAM template (scheduled-maintenance app)

Copy and paste the following code into a file named `template.yaml`.

```
AWSTemplateFormatVersion: '2010-09-09'
Transform: AWS::Serverless-2016-10-31
Description: SAM Template for Lambda function and EventBridge Scheduler rule

Resources:
  MyLambdaFunction:
    Type: AWS::Serverless::Function
    Properties:
      FunctionName: ScheduledDBMaintenance
      CodeUri: ./
      Handler: lambda_function.lambda_handler
      Runtime: python3.11
      Architectures:
        - x86_64
      Events:
        ScheduleEvent:
          Type: ScheduleV2
          Properties:
            ScheduleExpression: cron(0 3 1 * ? *)
            Description: Run on the first day of every month at 03:00 AM
      Policies:
        - CloudWatchLogsFullAccess
        - Statement:
            - Effect: Allow
              Action:
                - dynamodb:Scan
                - dynamodb:BatchWriteItem
              Resource: !Sub 'arn:aws:dynamodb:${AWS::Region}:${AWS::AccountId}:table/
MyOrderTable'

  LambdaLogGroup:
    Type: AWS::Logs::LogGroup
    Properties:
      LogGroupName: !Sub /aws/lambda/${MyLambdaFunction}
      RetentionInDays: 30

Outputs:
  LambdaFunctionName:
    Description: Lambda Function Name
    Value: !Ref MyLambdaFunction
```

```
  LambdaFunctionArn:
    Description: Lambda Function ARN
    Value: !GetAtt MyLambdaFunction.Arn
```

> **Note**
>
> AWS SAM templates use a standard naming convention of `template.yaml`. In this example, you have two template files - one to create the example database and another to create the app itself. Save them in separate sub-directories in your project folder.

This AWS SAM template defines the resources for your app. We define the Lambda function using the `AWS::Serverless::Function` resource. The EventBridge Scheduler schedule and the trigger to invoke the Lambda function are created by using the `Events` property of this resource using a type of `ScheduleV2`. To learn more about defining EventBridge Scheduler schedules in AWS SAM templates, see ScheduleV2 in the *AWS Serverless Application Model Developer Guide*.

In addition to the Lambda function and the EventBridge Scheduler schedule, we also define a CloudWatch log group for your function to send records of deleted items to.

Test script

Copy and paste the following code into a file named `test_app.py`.

```
import boto3
from datetime import datetime, timedelta
import json

# Initialize the DynamoDB client
dynamodb = boto3.resource('dynamodb')

# Specify your table name
table_name = 'YourTableName'
table = dynamodb.Table(table_name)

# Get the current date
current_date = datetime.now()

# Calculate the date one year ago
one_year_ago = current_date - timedelta(days=365)
```

```
# Convert the date to string format (assuming the date in DynamoDB is stored as a
 string)
one_year_ago_str = one_year_ago.strftime('%Y-%m-%d')

# Scan the table
response = table.scan(
    FilterExpression='#date < :one_year_ago',
    ExpressionAttributeNames={
        '#date': 'Date'
    },
    ExpressionAttributeValues={
        ':one_year_ago': one_year_ago_str
    }
)

# Process the results
old_records = response['Items']

# Continue scanning if we have more items (pagination)
while 'LastEvaluatedKey' in response:
    response = table.scan(
        FilterExpression='#date < :one_year_ago',
        ExpressionAttributeNames={
            '#date': 'Date'
        },
        ExpressionAttributeValues={
            ':one_year_ago': one_year_ago_str
        },
        ExclusiveStartKey=response['LastEvaluatedKey']
    )
    old_records.extend(response['Items'])

for record in old_records:
    print(json.dumps(record))

# The total number of old records should be zero.
print(f"Total number of old records: {len(old_records)}")
```

This test script uses the AWS SDK for Python (Boto3) to create a connection to your DynamoDB table and scan for items older than one year. To confirm if the Lambda function has run successfully, at the end of the test, the function prints the number of records older than one year still in the table. If the Lambda function was successful, the number of old records in the table should be zero.